Report for Question 4 in Deep Learning Coursework

Data Processing

The first step is to filter the original data frame to construct a customised dataset. For the training and testing sets, a dictionary is created to map the original item IDs to consecutive indices, including a padding token (0). This is necessary because the raw item IDs are extremely large and cannot be directly used for efficient model training. For each session, we adopt a next-item prediction setting. Specifically, all clicks except the last one are used as input sequences, while all clicks except the first one are treated as target outputs. In the padding step, all sequences are extended to the same length by matching the longest sequence in the dataset. Finally, the resulting data are shuffled and batched for model training.

Experiments

The benchmark experiments setting consists: epochs: 5; loss function: sparse categorical entropy; optimizer: Adam with learning rate 1e-3; neural network: one Input layer with dimension `(None, )`, one Embedding layer with dimension `(num class, 64)`, one Dropout layer with rate 0.1, and one GRU layer with output dimension 64. And benchmark metrics are displayed as:

Test Performance

Test loss: 4.1458

k	Recall@k	MRR@k	NDCG@k
5	0.3264	0.1933	0.2262
20	0.5635	0.2172	0.2944

Discussion

Although the performance improves as k increases, the overall results remain limited. In particular, the relatively low MRR and NDCG values indicate that relevant items are not consistently ranked at top positions. This suggests that the benchmark model struggles to capture users' preferences effectively in this recommendation task.

The attempt to the improvement of this model focuses on two aspects: tuning the hyperparameter and adjusting the structure of the neural network.

Hyperparameter tuning

The adjustment of hyperparameters focuses on different dropout rates (0.1/0.2/0.3), output dimensions of GRU layer and Embedding layers (64/128/256), and batch size for the training and testing datasets (64/256).

Type	Setting	Train Loss	Test Loss	$k = 5$			$k = 20$		
				Recall	MRR	NDCG	Recall	MRR	NDCG
Benchmark	DEFAULT	4.1009	4.1458	0.3264	0.1933	0.2262	0.5635	0.2172	0.2944
Dropout	0.2	4.1009	4.1525	0.3181	0.1845	0.2176	0.5506	0.2085	0.2851
Dropout	0.3	4.1105	4.0885	0.3248	0.1918	0.2248	0.5645	0.2162	0.2939
Out Dim	128	4.0756	4.0618	0.3238	0.1897	0.2229	0.5464	0.2123	0.2871
Out Dim	256	4.0698	4.1891	0.2998	0.1737	0.2050	0.5265	0.1970	0.2706
Learning Rate	1×10^{-5}	4.4098	4.4653	0.1219	0.0670	0.0807	0.2027	0.0754	0.1041
Learning Rate	1×10^{-4}	4.1408	4.2992	0.3334	0.2022	0.2348	0.5375	0.2234	0.2942
Batch Size	256	5.1259	5.0859	0.3840	0.2334	0.2708	0.6135	0.2573	0.3375

Table 1: Overall Performance Comparison under Different Hyperparameter Settings

Table 1 shows that variations in the dropout rate and output dimension do not lead to significant changes in overall model performance. However, when the output dimension is increased

to 256 for both the embedding and GRU layers, the test loss becomes noticeably higher than the training loss, indicating the presence of overfitting. In terms of batch size, increasing the batch size not only accelerates the training process but also slightly improves the model's ranking performance.

The choice of output dimension is motivated by the fact that higher-dimensional representations may capture more informative features. However, excessively large dimensions may also increase the risk of overfitting. In terms of batch size, the benchmark model adopts a relatively small batch size, which leads to slow training on large-scale datasets. Therefore, larger batch sizes are explored to improve training efficiency and and leads to improved model performance.. Regarding the learning rate, it is deliberately reduced to prevent unstable optimization and poor convergence. However, when the learning rate is too small, the model converges slowly and exhibits underfitting, as indicated by the relatively high training loss.

The structure of the neural network

Several alternative architectures, such as LSTM and bidirectional recurrent layers, are also explored to further improve the proposed model. Additional experiments are conducted by combining different types of recurrent neural networks to evaluate their impact on performance.

Model	Train Loss	Test Loss	$k = 5$			$k = 20$		
			Recall	MRR	NDCG	Recall	MRR	NDCG
Benchmark (GRU)	4.1009	4.1458	0.3264	0.1933	0.2262	0.5635	0.2172	0.2944
Mix RNN	4.0951	4.7958	0.3124	0.1868	0.2179	0.5457	0.2106	0.2853
LSTM	4.0933	4.0727	0.3281	0.1959	0.2285	0.5530	0.2188	0.2934
Bi-GRU	4.7036	4.5501	0.7414	0.6765	0.6927	0.8229	0.6849	0.7163

Table 2: Comparison of Different Recurrent Architectures Including the Benchmark Model

Apart from the Bi-GRU model, none of the remaining architectures shows substantial improvement over the benchmark. Notably, the mixed RNN model, composed of an LSTM layer followed by a GRU layer, suffers from severe overfitting. Among all evaluated models, the Bi-GRU architecture achieves **the best performance**. Its core component is a bidirectional GRU layer with an output dimension of 32, which enables more effective modelling of sequential dependencies.

The motivation for adopting LSTM lies in its ability to capture long-term dependencies through its memory mechanism. In this task, LSTM is expected to preserve more information from previous states, thereby improving sequential modelling beyond short-term transitions. The bidirectional architecture is motivated by a similar idea, but further incorporates future contextual information to enhance the inference process in an offline setting.

Future Work

Given more time, further experiments could be conducted using attention-based or Transformer-based architectures, which have demonstrated strong performance in session-based recommendation and address some limitations of traditional RNN architectures. Moreover, alternative loss functions that are more closely aligned with ranking metrics, such as pairwise ranking losses, may further improve performance. Additional improvements could also be achieved by incorporating side information and applying more advanced training strategies, such as early stopping, as the current experiments adopt only five training epochs.